

AURIX RAG Node: Architecture & Requirements

Assignee: Bhavya (Backend & Infrastructure) **Context:** This document outlines the required capabilities for the Supabase RAG (Retrieval-Augmented Generation) node based on real-world testing of the LangGraph AI pipeline.

1. The Core Problem (Why we need this ASAP)

During testing of the `gpt-oss-120b` model, the AI engine successfully found 13 vulnerabilities but only managed to write working exploits and patches for **2 of them**.

To fix this, we manually added complex "Few-Shot Examples" (templates for SSRF and Pickle Deserialization) directly into the Red Agent's system prompt.

- **The Result:** It worked! The engine successfully neutralized a 3rd vulnerability (SSRF in `urllib`).
- **The Flaw:** LLMs are stateless. If we try to scale this by hardcoding all 50+ OWASP vulnerability templates into the prompt, the prompt size will explode to thousands of tokens. Because LangGraph runs a state loop, we would be paying for those thousands of tokens on *every single node execution* across every single finding. This would skyrocket API costs and drastically slow down the Time-To-First-Token (TTFT) inference speed.

2. The RAG Solution

Instead of a massive, static prompt, we need the LangGraph orchestrator to start with a tiny prompt. When the engine detects a specific bug (e.g., `avoid-pickle`), it will query the RAG node. The RAG node will fetch *only* the specific Python template for exploiting and patching Pickle deserialization and inject it into the prompt.

3. Minimum Capabilities Required

Bhavya, please ensure the Supabase vector database and the RAG API endpoints support the following minimum requirements:

A. The Knowledge Base (Supabase / pgvector)

You need to seed the vector database with specific, structured data:

- **Vulnerability Templates:** Python/JS examples of how to exploit and patch specific bug types (e.g., SQLi, XXE, SSRF, Deserialization, Command Injection).
- **OWASP Guidelines:** Summarized mitigation tactics from the OWASP Cheat Sheet Series.

B. The API Interface

Divyansh's LangGraph node will need a fast, internal API endpoint to call.

- **Input:** A search query containing the scanner's `rule_id`, `title`, and language (e.g., `{"query": "python.lang.security.deserialization.pickle.avoid-pickle.python"}`).
- **Output:** A JSON array of the Top 2 or 3 most relevant markdown text chunks.

C. Performance Constraints

Because this RAG query happens *before* the Red Agent starts thinking, it sits directly in the critical execution path.

- **Latency:** The vector retrieval must happen in **under 500ms**.
- **Relevance:** It must use high-quality embeddings (like `text-embedding-3-small` or local `bge-small`) so it doesn't accidentally return an SQL injection template when the AI is trying to fix an XXE flaw.

4. Required Data Types for Ingestion

To ensure the LangGraph agents (Red and Blue) have exactly what they need to write successful exploits and patches, the RAG node must be capable of ingesting, indexing, and serving the following specific data types:

1. Vulnerability PoC Templates (Python/JS):

- *Example:* AST parsing scripts for detecting SSRF in `urllib` or Regex scripts for detecting unsafe `pickle.loads()`.
- *Purpose:* Teaches the Red Agent exactly how to programmatically prove a bug exists in a sandboxed environment without hallucinating syntax.

2. Patch Code Templates (Blue Agent):

- *Example:* Safe wrappers like `safe_urlopen(url)` or parameterized query templates for SQLAlchemy.
- *Purpose:* Provides the Blue Agent with secure coding patterns that are guaranteed to pass the sandbox tests.

3. OWASP Mitigation Guidelines:

- *Example:* Markdown summaries from the OWASP Cheat Sheet Series (e.g., "How to prevent XXE in lxml").
- *Purpose:* Provides the Blue Agent with the theoretical "why" and "how" of a mitigation strategy before it writes the code.

4. Historical CVE Fixes (Optional but highly recommended):

- *Example:* JSON/Markdown pairs of [Vulnerable Code Snippet] -> [Patched Code Snippet] from popular open-source repositories.
- *Purpose:* Gives the LLM deep contextual understanding of how real humans fixed similar bugs in the wild.

5. Language-Specific AST & Regex Guides:

- *Example:* "How to use Python's `ast.walk` to find `subprocess.Popen`".
- *Purpose:* The Red Agent relies heavily on Python's Abstract Syntax Tree (AST) to validate bugs without actually exploding the sandbox. It needs cheat sheets on how to parse code trees accurately.